

Security Audit

Bundle (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	9
Intended Smart Contract Functions	10
Code Quality	12
Audit Resources	12
Dependencies	12
Severity Definitions	13
Status Definitions	14
Audit Findings	15
Centralisation	19
Conclusion	20
Our Methodology	21
Disclaimers	23
About Hashlock	24

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Bundle team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Let's Bundle is a Web3 platform that empowers creators and communities to aggregate, package, and monetize digital assets into bundled offerings. By leveraging decentralized technology and smart contracts, it streamlines the process of combining NFTs, tokens, or other digital products into value-added packages—making distribution more efficient and engaging for both sellers and collectors.

Project Name: Bundle

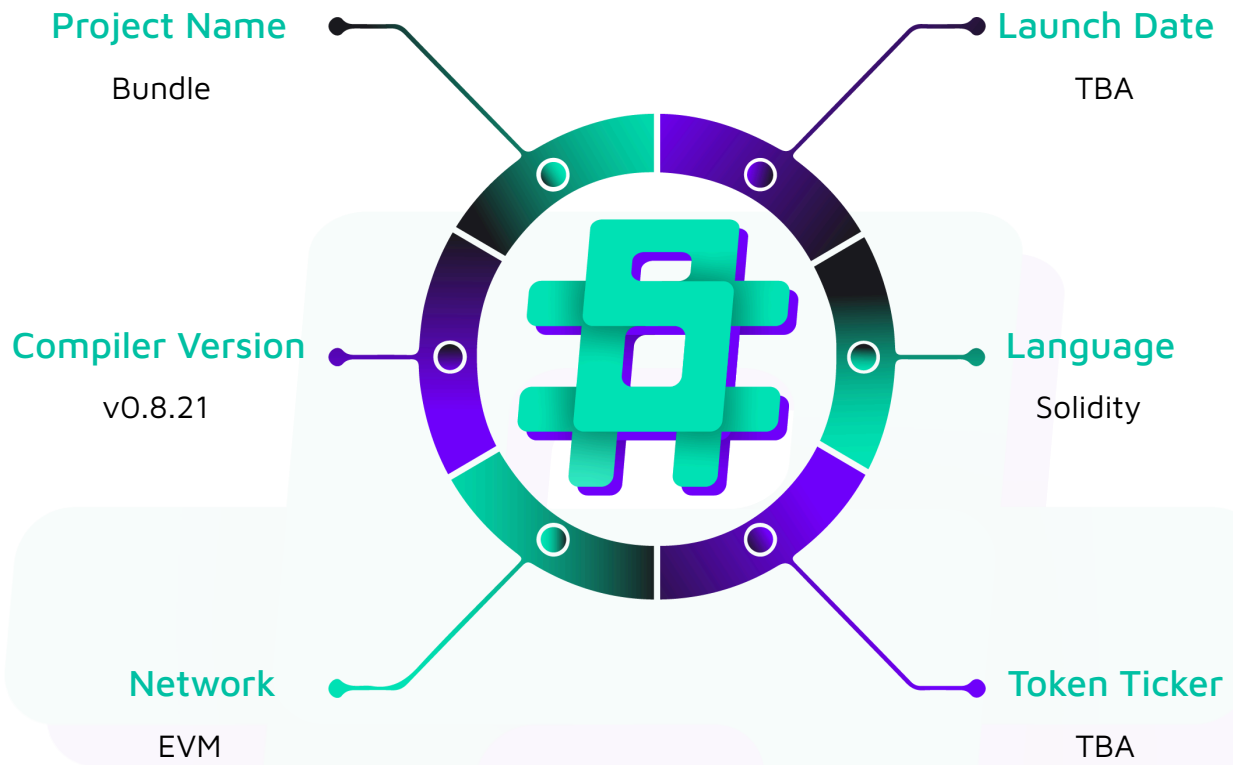
Project Type: Defi

Compiler Version: 0.8.21

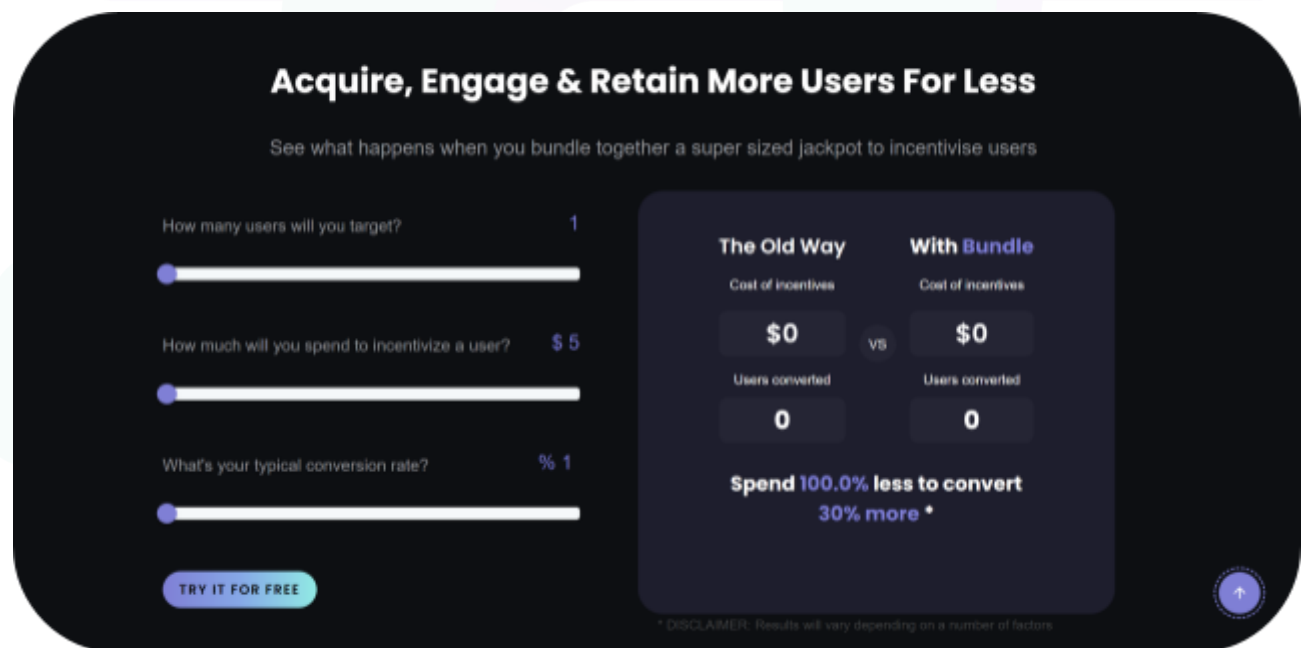
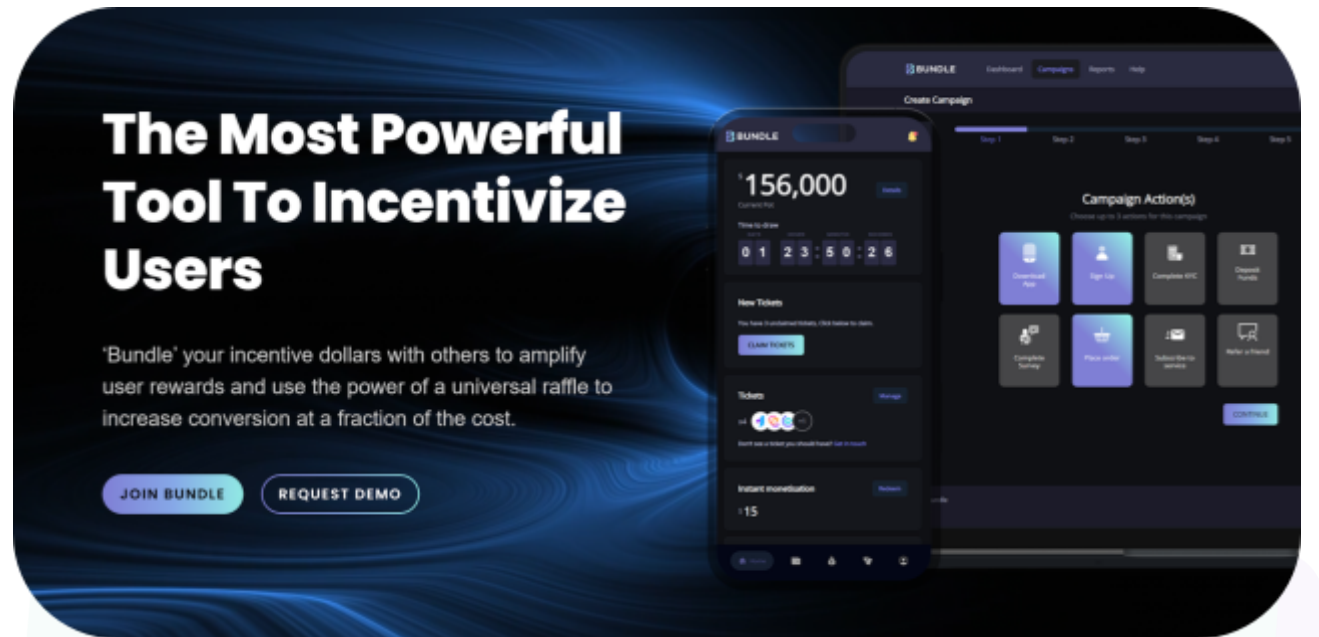
Website: www.letsbundle.io

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Bundle project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Bundle Smart Contracts
Platform	EVM / Solidity
Audit Date	February, 2025
Contract 1	BundleCampaign.sol
Contract 1 MD5 Hash	6286a6ea8f10522b490c5ba5612b3f6b
Contract 2	BundleFactory.sol
Contract 2 MD5 Hash	4edb85ed153d18333cb03104f2e5525d
Contract 3	BundleOrganization.sol
Contract 3 MD5 Hash	ff1e44a32029d9f7468d4997ac83a364
Contract 4	BundlePassport.sol
Contract 4 MD5 Hash	f63b0c7358d2450615c5965f2d4760d1
Contract 5	BundleTicket.sol
Contract 5 MD5 Hash	d3acae573ef5a3f463fdb43df681163c
Contract 6	Lottery.sol
Contract 6 MD5 Hash	f105a2ddcb41fc8f2803e80e2987dfb6
Contract 7	RandomGetter.sol
Contract 7 MD5 Hash	1381852a986e03221b56556e450a8ee1
Contract 8	TicketManager.sol
Contract 8 MD5 Hash	1ce60c22965352918ee4b2c9f3e5367a
Contract 9	TicketRedemption.sol

Contract 9 MD5 Hash	9a37fcdf10de5c4bd0bb3f8da9c97144
----------------------------	----------------------------------



Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We initially identified some vulnerabilities that have since been addressed.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The general security overview is presented in the [Standardised Checks](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 Medium severity vulnerability

1 Low severity vulnerability

3 Gas Optimisations

1 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
BundleCampaign.sol Manages the creation and ownership of campaign tokens within the Bundle platform	Contract achieves this functionality.
BundleFactory.sol Serves as a factory for various contracts such as organizations, campaigns, lotteries, redemption and tickets	Contract achieves this functionality.
BundleOrganization.sol Handles the creation and ownership of organization tokens in Bundle	Contract achieves this functionality.
BundlePassport.sol Handles the creation and ownership of passport tokens which own organizations	Contract achieves this functionality.
BundleTicket.sol Manages the minting and burning of ticket within the Bundle platform	Contract achieves this functionality.
Lottery.sol Manages lottery functionality including ticket registration, initialization, running, and rewarding winners	Contract achieves this functionality.
RandomGetter.sol Gets random numbers for lotteries using chainlink VRF	Contract achieves this functionality.
TicketManager.sol Manages the creation and transfer of tickets for	Contract achieves this functionality.

campaigns on the Bundle platform	
TicketRedemption.sol Enable users to exchange tickets for rewards on Bundle platform before the burn deadline	Contract achieves this functionality.



Code Quality

This audit scope involves the smart contracts of the Bundle project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Bundle project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] Lottery#rewardWinners - Unbounded loops when iterating over organizations from the organizations could cause out of gas error

Description

Because the Lottery contract allows for an unlimited number of organizations to be added, when the rewardWinners function iterates over the organizations during rewards handling, the contract may revert when a user tries to interact with the contract.

Vulnerability Details

The rewardWinners function loops over all organizations;

```
for (uint o; o < organizations.length; o++) {  
    ...  
}
```

Depending on the number of organizations that the contract supports, calls to rewardWinners may run out of gas, resulting in a denial of service of all external-facing functions.

Impact

Winners could never be rewarded.

Recommendation

It's recommended that the Lottery contract allows for the specification of an offset and length so that rewards can be handled in batches, or add a limited number of organizations to be added.

Status

Resolved

Low

[L-01] TicketRedemption#setRewardToken - Missing event emission

Description

The `setRewardToken` function does not emit an event to log critical state changes or actions.

Recommendation

It is recommended to add a relevant event in the function.

Status

Resolved

Gas

[G-01] BundleCampaign, BundleOrganization, Lottery, BundleTicket, TicketManager, TicketRedemption - State variables that could be made immutable

Description

The following variables are only set in the constructor.

```
address public lottery; // BundleCampaign.sol
address public redemption; // BundleCampaign.sol
address public organization; // BundleCampaign.sol
address public passport; // BundleOrganization.sol
uint32 public mintDeadline; // Lottery.sol
uint32 public burnDeadline; // Lottery.sol
uint32 public lotteryTime; // Lottery.sol
IRandomGetter public randomGetter; // Lottery.sol
address public campaign; // BundleTicket.sol
```

```
IBundleFactory public factory; // TicketManager.sol
address public lottery; // TicketRedemption.sol
```

Recommendation

Make the variables immutable.

Status

Resolved

[G-02] Lottery#_rewardOverCapWinner - Cache array length before looping

Description

Before iterating through an array with a for-loop, the length of the array can be cached in memory so that its value does not need to be accessed on each iteration of the loop.

```
for (uint256 i = 0; i < overCapWinners.length; i++) {
    ...
}
```

Recommendation

Cache the array length by assigning it to a variable in memory before looping through the array.

Status

Resolved

[G-03] BundlePassport#mintTo, BundleTicket#_mintToCampaign - State variables that are used multiple times in a function should be cached

Description

The above functions read `_lastTokenId` value from storage multiple times.

When performing multiple operations on a state variable in a function, it is recommended to cache it first. Either multiple reads or multiple writes to a state variable can save gas by caching it on the stack.



Recommendation

Cache the storage value into a memory and use it.

Status

Resolved

QA**[Q-01] Contracts - Inconsistent use of `uint` and `uint256` types****Description**

The contracts inconsistently use both `uint` and `uint256` data types for variable declarations. In Solidity, `uint` is an alias for `uint256`, meaning they are functionally equivalent. However, mixing these two notations can lead to confusion and reduce code readability. Consistency in data type usage is essential for maintaining clarity and ensuring that developers can easily understand the codebase.

Recommendation

It is recommended to standardize the use of data types throughout the contract.

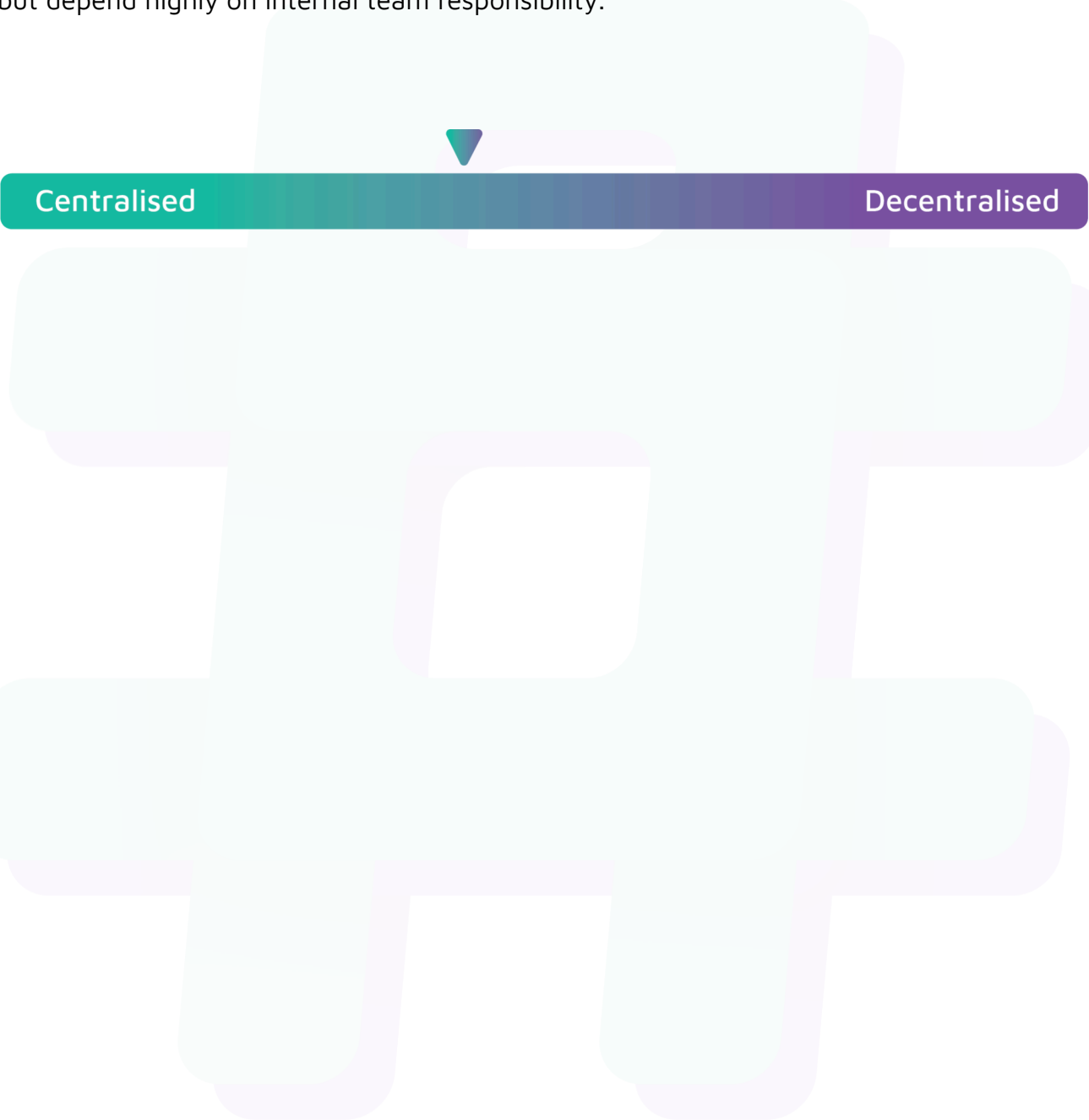
Status

Resolved

Centralisation

The Bundle project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Bundle project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.